

Memory mock autotests — setup and run

This section describes how to bring up a local Omega container running with the deterministic LLM mock and run the `test*_mock.py` suite in `Autotests/mock_memory/` against it. The container, image, IRC channel, and LLM mock controller are identical to those used by `Autotests/mock/`; only the pytest target directory differs.

1. Prerequisites

- Docker engine on the host.
- Repository checked out, working from its root.
- Python virtual environment under `Autotests/venv` with `pytest` installed.

2. Build the local image

The mock infrastructure is part of the source tree, so the image must be built locally rather than pulled from the registry.

```
docker build -t omega:mock .
```

3. Start the container with the Test provider

The container connects back to the host on TCP port 9765 to reach the mock LLM controller. `TEST_API_KEY` must hold the host IP that is reachable from inside the container. Under the default Docker bridge this is `172.17.0.1`.

```
docker run -d -it \  
  --name omega_mock \  
  --user 65534:65534 \  
  --init \  
  --network bridge \  
  --security-opt no-new-privileges:true \  
  --volume omega-mock-memory:/PeTTa/repos/Omega/memory/ \  
  --tmpfs /tmp:size=64m,mode=1777,exec \  
  --tmpfs /run:size=16m,mode=755 \  
  --tmpfs /var/tmp:size=64m,mode=1777,exec \  
  -e TEST_API_KEY=172.17.0.1 \  
  -e OMEGA_AUTH_SECRET=0000 \  
  omega:mock \  
  IRC_channel="#omega_mock" \  
  provider="Test" \  
  embeddingprovider="Local"
```

Notes:

- `provider="Test"` selects the mock LLM dispatcher.
- `embeddingprovider="Local"` keeps the embedding model in-process (no network call).
- `TEST_API_KEY=172.17.0.1` is the host's docker-bridge address.
- `OMEGA_AUTH_SECRET=0000` matches the value the test harness sends as `auth 0000`.
- The IRC channel can be any unique string; pick one not used by another concurrent run.

Wait until the container has registered on IRC and joined the channel:

```
docker logs omega_mock 2>&1 | grep -E "Joining|Registered"
```

4. Configure the test environment

Export the variables the test harness reads. The values must match the container name and IRC channel chosen at step 3.

```
export OMEGA_CONTAINER=omega_mock
export OMEGA_IRC_CHANNEL="#omega_mock"
```

5. Run the suite

```
cd Autotests
source venv/bin/activate
pytest -s -v mock_memory/test*_mock.py
```

The mock controller is provided by a session-scoped fixture in `mock_memory/conftest.py`, which reuses the `LlmMockController` from `Autotests/mock/llm.py` (the same controller used by the `mock/` suite). It is started once per pytest session. Expected output: 7 passed.

6. Tear down

```
docker rm -f omega_mock
docker volume rm omega-mock-memory
```

Tests description

All seven tests exercise the agent's memory machinery (working memory via `pin`, long-term memory via `remember`, history window, episodes lookup) using deterministic mock-LLM answers. Each test sends one or more prompts via IRC, registers a fixed answer for each prompt, and verifies the resulting skill calls and side effects (`history.metta`, ChromaDB, docker logs).

1. test_memory_pin_window_visibility_mock.py

Two-turn flow that verifies a `(pin ...)` block lands on disk in `history.metta` and remains inside the trailing 30K-byte HISTORY window so the agent can see it on the next iteration.

- Turn 1 mock answer: `(pin "<marker>") (send "Pinned <marker>.")`.
- Turn 2 mock answer: a short `(send ...)` acknowledgement.
- Checks: `(pin ...)` was invoked with the marker, the pin block is grep-able in `history.metta`, and the marker is present in the last 30000 bytes of the file.

2. test_pin_invisible_within_iteration_mock.py

Negative test that confirms a `(pin ...)` emitted inside iteration N is not present in the same iteration's PROMPT (the prompt is built before skill evaluation), and only enters HISTORY on iteration N+1.

- Mock answer: `(pin "<marker>") (send "Pinned a progress code.")`. The marker is intentionally not placed in the `HUMAN_MESSAGE` body so it cannot leak into the current iteration's PROMPT via that path.
- Checks (via docker logs): the `CHARS_SENT` line for the iteration that carries `REQ-<run_id>` must not contain the marker; at least one later `CHARS_SENT` line must contain it.

3. test_memory_history_byte_window_truncation_mock.py

Two-turn flow that exercises the sliding-window mechanic of `history.metta`: an early marker is written, then ~35K bytes of padding push it past the trailing 30K-byte HISTORY window. The marker stays in the file on disk but disappears from the slice fed back to the agent as HISTORY.

- Turn 1 mock answer: `(send "<early_marker>")`.
- Turn 2 mock answer: `(remember "<padding_marker>+~35K bytes of A") (send "padded")`.
- Checks: the early marker is initially inside the last 30000 bytes; after padding, the file grew by at least 35000 bytes; the early marker is still grep-able in the full file; the early marker is no longer inside the last 30000 bytes.

4. test_transition_episodes_after_eviction_mock.py

Three-turn flow that confirms an evicted marker is still recoverable via the `episodes` skill (timestamp-based scan of `history.metta`).

- Turn 1 mock answer: `(send "<BEACON>")`; the host captures `seed_time` immediately before sending.
- Turn 2 mock answer: `(remember "<padding>+~35K bytes") (send "padded")`, which evicts BEACON from the trailing 30K-byte window.
- Turn 3 mock answer: `(episodes "<seed_time>") (send "Recalled <BEACON>.")`.
- Checks: BEACON evicted from the last 30000 bytes; `(episodes ...)` was invoked with a prefix matching the captured seed timestamp; the final `(send ...)` references the BEACON marker.

5. test_transition_pin_to_remember_mock.py

Two-turn flow that promotes a working-memory item to long-term memory: turn 1 pins three candidates; turn 2 commits the same set via `remember`. ChromaDB grows by exactly one vector.

- Turn 1 mock answer: `(pin "<marker>: candidates A, B, C") (send "Pinned <marker>: A, B, C.")`.
- Turn 2 mock answer: `(remember "<marker>: candidates A, B, C") (send "Committed <marker> to long-term memory.")`.
- Checks: both skill calls landed with the same marker, and `chroma.sqlite3` embeddings count rose from N to N+1.

6. test_transition_metta_to_remember_mock.py

Single-turn flow that combines an in-process reasoning step with persistence: the agent calls `metta` for NAL inheritance and then `remember` for the conclusion. ChromaDB grows by exactly one vector.

- Mock answer:
 - `(metta "(|- ((-> sam friend) (stv 1.0 0.9)) ((-> garfield animal) (stv 1.0 0.9)))")`
 - `(remember "<conclusion_marker>: Sam is friend of an animal (derived via NAL inheritance).")`
 - `(send "Reasoned and remembered.")`
- Checks: `(metta ...)` was invoked; `(remember ...)` was invoked with the conclusion marker; `chroma.sqlite3` embeddings count rose from N to N+1.

7. test_last_skill_results_visible_next_turn_mock.py

Verifies the `LAST_SKILL_USE_RESULTS` carry: results of skill calls in iteration N appear in the assembled PROMPT for iteration N+1.

- Mock answer: `(metta "(quote <sentinel>") (send "computed"))`. The sentinel is placed inside the `metta` expression so it can be located in the next iteration's PROMPT.
- Checks (via docker logs): the `CHARS_SENT` line that follows the one carrying `REQ-<run_id>` contains the `LAST_SKILL_USE_RESULTS` marker and the sentinel string.